Multi-Agent Workspace Assistant

A choreography-based AI assistant for Google Workspace

Course: Python Programming for Engineers

Team — Group 3

- Trần Tiến Cường *(Team Leader)*
- Nguyễn Trần Quốc Tính

Task Allocation

Member	Responsibilities	Contribution
Trần Tiến Cường *(Team Leader)*	System architecture and choreography design; API Gateway; Intent Gate; Redis Blackboard and graph memory; LangGraph state-machine logic; shared library; Tracer; web frontend; and the cloud infrastructure (Terraform provisioning, Cloud Build, and deployment).	**50%**
Nguyễn Trần Quốc Tính	The domain worker agents — Gmail, Drive, Calendar, and Docs — and their integration with the corresponding Google Workspace APIs; researching extensibility patterns and designing integration interfaces for potential third-party platforms (e.g., Slack, Microsoft 365, Notion) in future system expansions.	**50%**

1. Introduction & Overview

Modern knowledge workers spend a large share of their day moving between Gmail, Google Drive, Google Calendar, and Google Docs — reading email, locating files, scheduling events, and drafting documents. Each task is simple on its own, but constantly switching tools and repeating manual steps adds up to significant friction.

This project, **Multi-Agent Workspace Assistant**, is a system that lets a user drive these Google Workspace services through natural language. Instead of clicking through four different apps, the user types a request such as *“summarize my unread emails from this week and draft a reply to the one from the professor”* — and the system interprets the intent, dispatches it to the right specialized agents, and returns the result.

The core idea is a **multi-agent choreography**: rather than one large monolithic AI, the system is built from several small, independent agents — one per Workspace domain — that coordinate by passing

messages over a shared bus, with no central orchestrator dictating every step. A lightweight **Intent Gate** classifies each request and routes it; the agents then react autonomously and publish their results to a shared state store.

Goals

- Provide a single conversational entry point to multiple Google Workspace services.
- Demonstrate a decoupled **choreography** architecture where agents can be added or removed independently.
- Support **human-in-the-loop (HITL)** confirmation for sensitive actions (e.g. sending an email).
- Persist context across turns through a **graph-based memory**.

Scope

The project focuses on the four Workspace domains above, a web frontend for the chat interface, and the supporting backend infrastructure (message broker, state store, database). It was deployed to Google Cloud for demonstration. Full production hardening — multi-tenant scaling and exhaustive error recovery — is out of scope and discussed under *Limitations & Future Work*.

2. System Architecture

The system follows a **choreography** model rather than orchestration. In an orchestration model, a central controller would explicitly call each service in turn. In choreography, every component instead *reacts* to messages on a shared broker and publishes its own results — there is no single point that knows the whole workflow. This keeps the agents fully decoupled: a new domain agent can be plugged in simply by subscribing to the broker, without modifying any existing component.

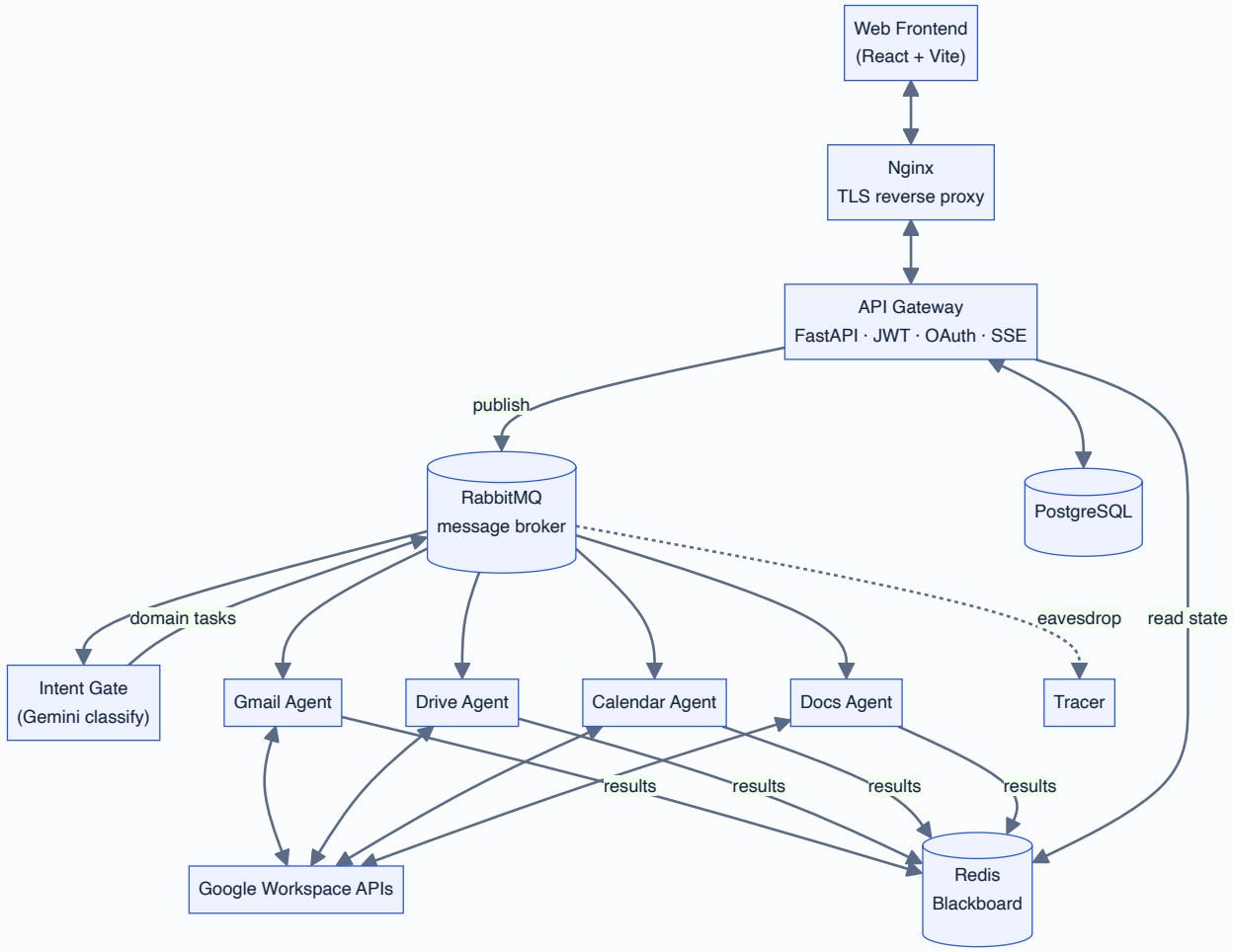

Figure 1 — High-level choreography architecture. Solid arrows are the request/result path; the dotted arrow shows the Tracer passively observing all broker traffic.

Architecture evolution (v4 → v5)

An earlier iteration (v4) coordinated agents through a shared `result-key` convention on the message payload. This proved brittle as workflows grew. The current design (v5) replaces it with two changes: each agent runs an internal **LangGraph finite-state machine (FSM)** to manage its own multi-step reasoning, and shared state moved to a **Redis Blackboard** that any component can read or update. The Blackboard acts as the single source of truth for an in-flight request, decoupling the producer of a result from its consumer.

3. Components & Services

The system is split into independent services, each packaged as its own container. A shared library holds the common code (message schemas, Blackboard helpers, database models) so that every service speaks the same protocol.

Service	Responsibility

API Gateway	Single HTTP entry point. Handles JWT authentication, Google OAuth, request validation, publishing user messages to the broker, and streaming results back to the browser over Server-Sent Events (SSE).
Intent Gate	Classifies each incoming request with a fast LLM (Gemini Flash) and publishes one or more typed tasks to the appropriate domain queues.
Gmail Agent	Search, read, summarize, draft, and send email via the Gmail API.
Drive Agent	Locate, list, and manage files in Google Drive.
Calendar Agent	Create, query, and manage calendar events.
Docs Agent	Read and draft Google Docs content.
Tracer	Passively subscribes to all broker traffic and records a trace of every message into Redis for debugging and the in-app activity log.
Shared library	Common message envelopes, Blackboard client, database schema, and utilities reused by every service.
Frontend	React + Vite chat interface with lazy-loaded tabs for each Workspace domain.

4. Key Mechanisms

4.1 Request lifecycle

A single user request flows through the system as a chain of asynchronous messages. The Gateway never waits synchronously for an agent; instead it streams results to the browser as they land on the Blackboard.

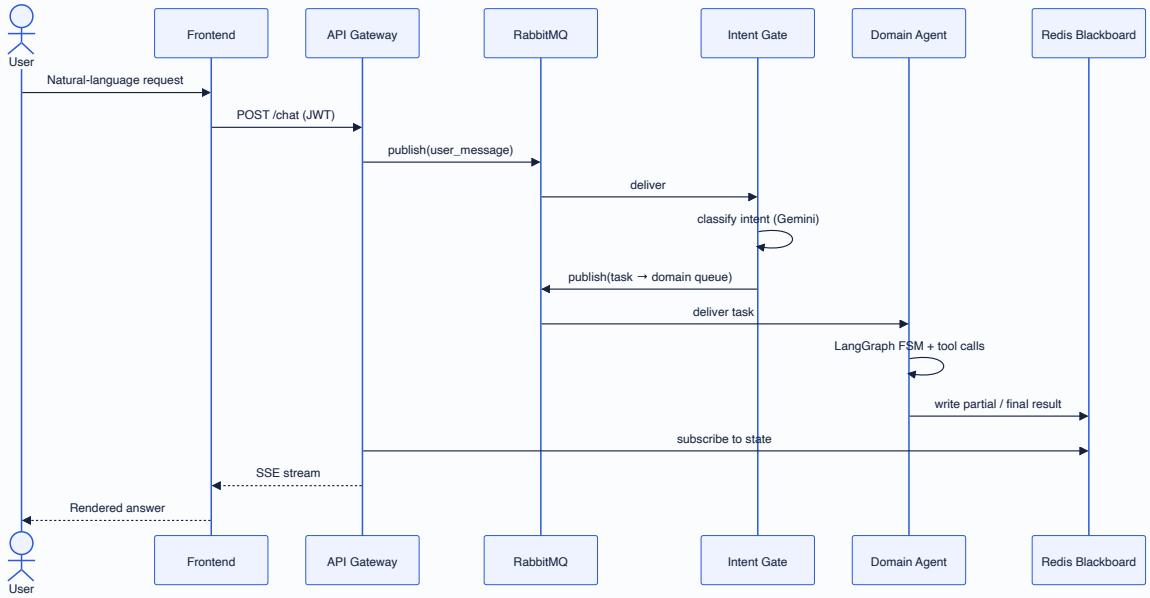

Figure 2 — Lifecycle of one request, end to end.

4.2 Agent reasoning as a finite-state machine

Each domain agent runs an internal **LangGraph FSM**. The agent plans, calls a tool (a Google API or the LLM), observes the result, and decides whether to continue, ask the user for confirmation, or finish. Modeling the agent as explicit states makes its behavior predictable and easy to trace.

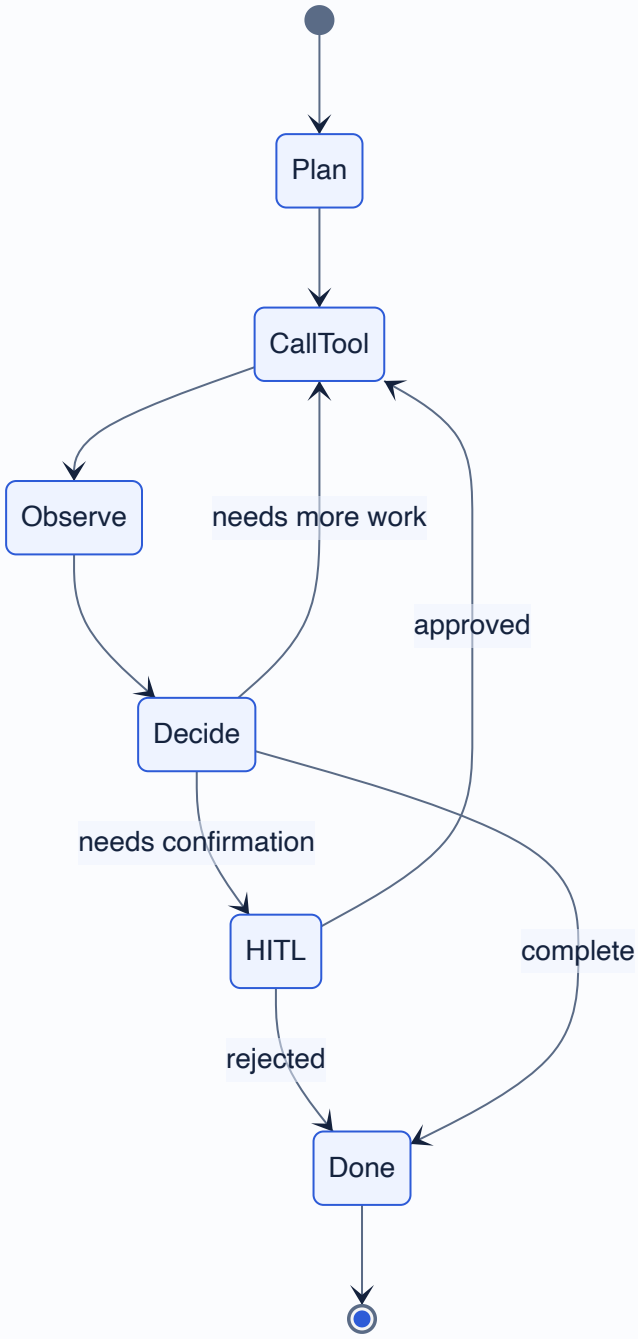

Figure 3 — Internal state machine of a domain agent.

4.3 Human-in-the-loop (HITL)

Sensitive actions — sending an email, creating a calendar event — are never executed silently. When an agent reaches such a step it emits a typed **"kind" envelope** describing the pending action. The Gateway surfaces this to the user as a confirmation prompt; only after the user approves does the agent resume and complete the action.

4.4 UI-tool vs. Agent-tool

The system distinguishes two ways a capability can be invoked. **Agent-tools** run through the full asynchronous choreography described above. **UI-tools** are direct, synchronous REST calls served by the Gateway for interactive features that need an immediate response — for example route/preview lookups and the Calendar Canvas view. Separating the two keeps the conversational path decoupled while still allowing snappy, synchronous UI interactions.

4.5 Graph memory

To remember context across turns, conversations are distilled into a **graph memory**. Entities and the relationships between them are extracted and stored as nodes and edges. A **relevance gate** filters out low-value information before it is written, which prevents the graph from exploding with noise — a common failure mode for naive memory systems. Existing embeddings are preserved on update so that semantic search stays stable.

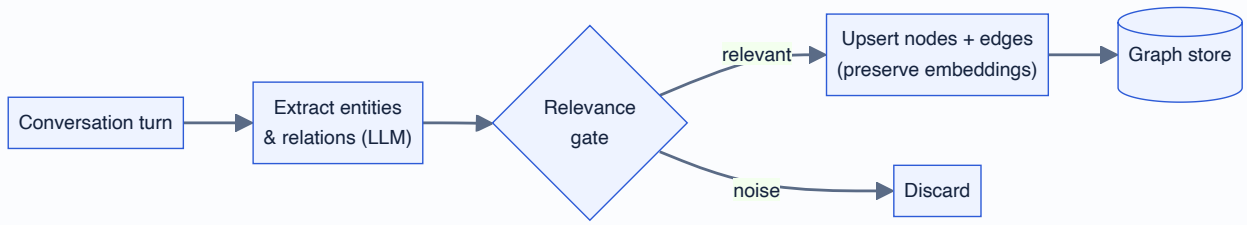

Figure 4 — Graph-memory ingestion pipeline.

5. Technology Stack & Infrastructure

Layer	Technology
Backend services	Python, FastAPI, LangGraph
LLM	Google Gemini (Flash-class model) via native REST
Message broker	RabbitMQ
State store	Redis (Blackboard + trace log)
Database	PostgreSQL (users, OAuth tokens, history, plans, memory)
Frontend	React, Vite, TypeScript
Edge / TLS	Nginx reverse proxy with origin certificates
Packaging	Docker & Docker Compose
Cloud	Google Compute Engine, provisioned with Terraform, built & deployed via Cloud Build

For the demonstration the whole stack ran on a single Compute Engine VM behind Nginx, with all backend containers on one Docker network and only ports 80/443 exposed publicly. The cloud

resources — VM, network, firewall, static IP, container registry and service accounts — were defined as code in Terraform, making the environment reproducible. After the demonstration period the infrastructure was fully torn down to avoid ongoing cost; it can be recreated from the Terraform definitions at any time.

6. Results

The implemented system successfully handles natural-language requests across all four Workspace domains through the single chat interface. Concretely, the assistant can:

- Search and summarize Gmail messages, then draft and (after confirmation) send replies.
- Locate and list Drive files on request.
- Create and query Calendar events, with an interactive Calendar Canvas view.
- Read and draft Google Docs content.
- Route mixed requests to the correct agent automatically via the Intent Gate.
- Pause for user confirmation on sensitive actions through the HITL flow.
- Carry context across turns using the graph memory.

The choreography design met its goal: each agent is an independent container that can be developed, deployed, and restarted on its own, and the Tracer provides full visibility into the message flow for debugging. The system was demonstrated end to end on the live Cloud deployment.

7. Limitations & Future Work

- **End-to-end & UI verification.** Automated coverage of the full multi-agent flow and the frontend is still incomplete; testing was largely manual.
- **Calendar event typing.** Finer-grained handling of calendar `event_type` variants remains to be implemented.
- **LLM output limits.** The memory pipeline does not yet cap `maxOutputTokens`, which can lead to oversized generations on long inputs.
- **Single-node deployment.** The demo ran on one VM; horizontal scaling and high availability were not addressed.
- **Secrets management.** Configuration secrets were managed manually for the demo; a managed secret store would be the production approach.

8. Conclusion

This project delivered a working multi-agent assistant that unifies Gmail, Drive, Calendar, and Docs behind a single conversational interface. Its central contribution is the **choreography architecture**: independent, message-driven agents coordinated through a shared Blackboard rather than a central

orchestrator, each running an explicit LangGraph state machine. Combined with human-in-the-loop confirmation and graph-based memory, the design proved both extensible and observable.

The main lesson was the value of decoupling. Moving from the brittle shared-key approach of v4 to the Blackboard-plus-FSM model of v5 made the system markedly easier to reason about and extend. The clearest avenues for future work are stronger automated verification and production-grade deployment concerns such as scaling and secret management.